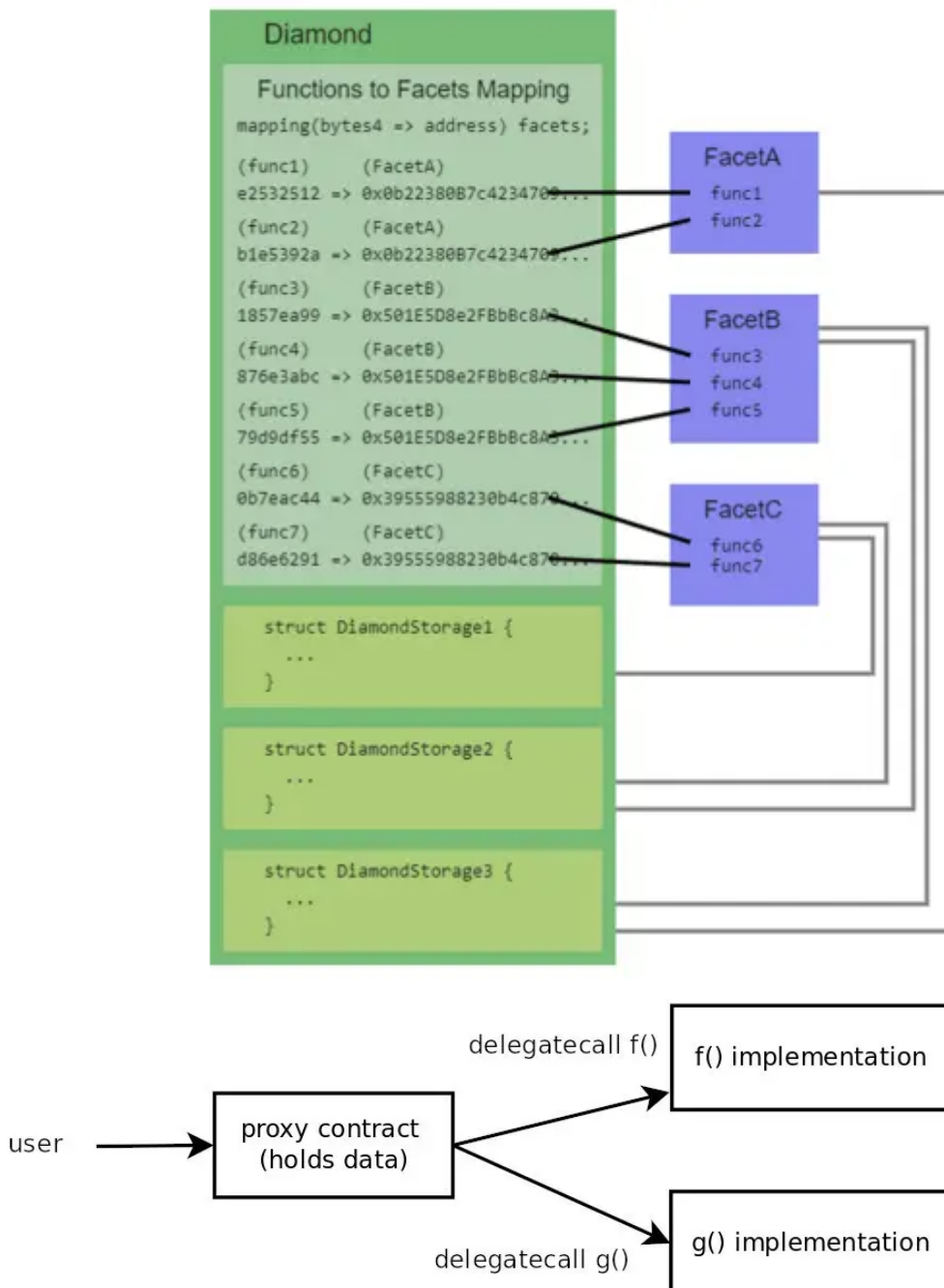


EIP-2535 钻石代理介绍及重构设计

EIP-2535 是以太坊上一个将合约进行代码模块化组合的提案，其目的是为了大型的智能合约突破 24kb 大小的最大限制，并且让合约更方便地更新功能。

- **钻石 (diamond)**：钻石可以理解为代理合约 (Proxy)，也是与用户进行交互的主合约
- **切面 (facet)**：正如真正的钻石有不同的侧面一样，一个钻石合约也有着不同的面，钻石合约的每个功能所需要调用的合约对应一个切面，所以也可以理解为实现合约 (Implementation)
- **钻石切割 (diamondCut)**：钻石协议标准扩展了一种叫钻石切割的功能，其主要作用从钻石中增加、替换或删除切面和功能，可以理解为合约的升级 (Upgrade)
- **放大镜 (The Loupe)**：钻石协议标准中的放大镜功能主要是返回关于切面的信息和钻石存在的功能，这些信息是保存在钻石合约内部的存储结构——DiamondStorage 中



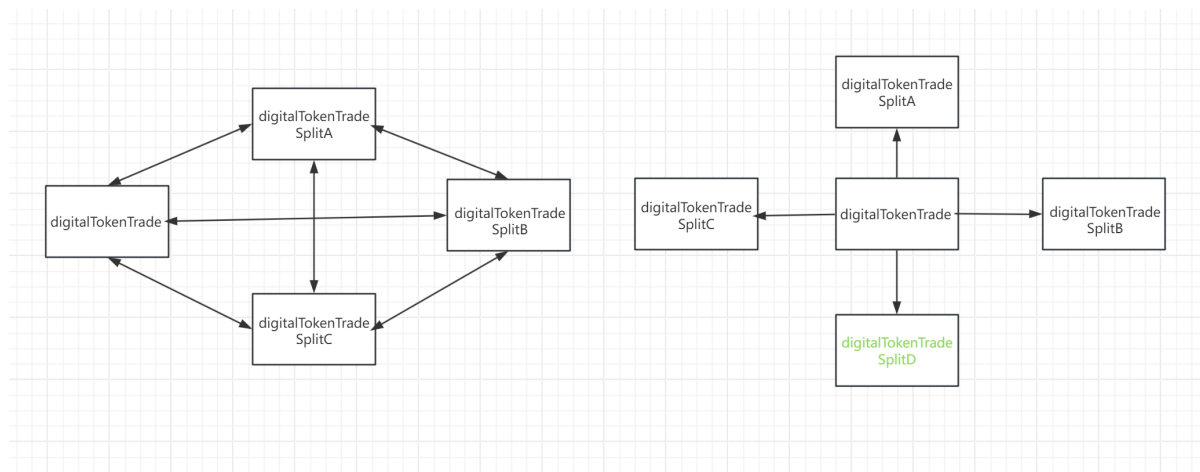
用户与代理合约交互，代理合约在fallback中使用了映射表根据要调用的函数（函数选择器）来选择合适的实现合约地址，向实现合约发送 delegatecall 调用实现合约内的函数。执行的是实现合约内的代码，但整套合约的 storage 保存在代理合约内。多个实现合约可以共享存储

当钻石代理合约被部署时，它必须将 DiamondCutFacet 合约的地址添加到钻石代理合约中

在部署或升级实现合约时，重新部署新的实现合约地址，并调用代理合约的函数添加、替换或删除这些函数选择器到实现合约的映射

```
function diamondCut(  
    FacetCut[] calldata _diamondCut,  
    address _init,  
    bytes calldata _calldata  
) external;
```

dtc合约重构前后合约关系:



存储管理

因多个实现合约共享代理合约的存储，需要正确管理存储槽以防止冲突，所有合约要以相同顺序声明相同的状态变量

DiamondStorage和AppStorage存储模式，分别用于隔离和共享存储

DiamondStorage

```
// A contract that implements Diamond Storage.
library LibA {

    // This struct contains state variables we care about.
    struct DiamondStorage {
        address owner;
        bytes32 dataA;
    }

    // Returns the struct from a specified position in contract storage
    // ds is short for DiamondStorage
    function diamondStorage() internal pure returns(DiamondStorage storage ds) {
        // Specifies a random position in contract storage
        // This can be done with a keccak256 hash of a unique string as is
        // done here or other schemes can be used such as this:
        // bytes32 storagePosition = keccak256(abi.encodePacked(ERC1155.interfaceId, ERC1155.name, address(this)));
        bytes32 storagePosition = keccak256("diamond.storage.LibA");
        // Set the position of our struct in contract storage
        assembly {ds.slot := storagePosition}
    }
}

// Our facet uses the Diamond Storage defined above.
contract FacetA {

    function setDataA(bytes32 _dataA) external {
        LibA.DiamondStorage storage ds = LibA.diamondStorage();
        require(ds.owner == msg.sender, "Must be owner.");
        ds.dataA = _dataA;
    }

    function getDataA() external view returns (bytes32) {
        return LibA.diamondStorage().dataA;
    }
}
}
```

AppStorage

将所有存储变量存放在AppStorage结构中

每个切面将AppStorage结构声明为第一个也是唯一——一个状态变量，位于存储槽的第0位。然后不同的切面可以从该结构中访问变量

在智能合约中向存储结构添加新的状态变量时，必须将其添加到结构的末端

```
// AppStorage.sol
struct AppStorage {
    uint256 secondVar;
    uint256 firstVar;
    uint256 lastVar;
    ...
}

import "./AppStorage.sol"

contract StakingFacet {
    AppStorage internal s;

    function myFacetFunction(uint256 _nextVar) external {
        s.total = s.firstVar + _nextVar;
    }
}
```

DTT合约重构设计

存储结构文件

digitalTokenTradeStorage.sol

```
struct DTTStorage {
    Config public config;
    address idFactoryAddress;
    // 交易相关存储
    mapping(string => RealisedTokenTrade) public businessIndex;
    mapping(string => RealisedTokenTradeExpand) public businessIndexExpand;

    // 条件相关存储
    mapping(string => ConditionSet) public conditionSetIndex;
    mapping(string => SingleCondition) public singleConditionIndex;
    SingleCondition[] scArray;
    ConditionSet[] csArray;
    mapping(string => Condition.SingleCondition) scMap
}

struct RealisedTokenTrade {
    address from;
    address to;
    address tokenAddr;
    uint256 amount;
    uint256 tradeTime;
    string timescId; // Condition ID for transaction time
    string conditionSetId; // Condition set ID for transaction operations
    bool partialAcceptEnable;
    address partialAcceptAddress;
    string partialAcceptScId; // Condition ID for partial acceptance
    string parentBusinessId;
    string[] subBusinessIds;
    SettleStatus status;
}

struct RealisedTokenTradeExpand {
    uint256 guaranteeAmount;
}

struct ConditionSet {
    string id;
    string[] scIDs;
    string[] csIDs;
    JoinType join;
}

struct SingleCondition {
    string id;
    string conditionType;
    string description;
    ConditionFactor[] fixFactors;
    ConditionFactor[] dynamicFactors;
}
```

```

struct ConditionFactor {
    string name; // Atomic type
    string value; // Atomic modification value
    bool changeFlag; // Has the atomic factor been modified
    bool changeAble; // Is it changeable
    address changeAddr; // Operator
    uint256 beginTime; // Operation start time
    uint256 endTime; // Operation end time
    string commentsHash;
    string[] filesHash;
}

```

新的存储变量要加入, 在digitalTokenTradeStorage.sol文件中DTTStorage结构最后加入新的存储变量, 另外新类型定义在文件最后而不是在DTTStorage里

因为使用AppStorage共享存储模式, 各Facet合约第一个也是唯一一个存储变量为DTTStorage结构的实例

```

import "./digitalTokenTradeStorage.sol"

contract AFacet {
    DTTStorage internal s;
    ...
}

```

Facet设计

将目前分为以下Facet

1. 交易发送相关方法
 - sendFacet.sol
 - sendRealisedToken
2. 交易条件操作相关方法
 - conditionActionFacet.sol
 - conditionAccept
 - conditionReject
 - conditionAction
 - conditionPartialAccept
 - conditionSetDate
3. 结算兑付相关方法
 - settleFacet.sol
 - settleTrade
 - settleTradeWithAmount
4. 交易状态管理相关方法
 - statusFacet.sol
 - tradeStatus
 - setTradeStatus

5. 交易条件创建和复制相关方法

conditionCreateFacet.sol
create
copy

6. 交易条件修改相关方法

conditionChangeFacet.sol
changeFactor

7. 交易条件查询相关方法

conditionQueryFacet.sol
queryCSStatus
querySCStatus
dateNotSet
timeRangeValidate
factorFutureChangeChance
calculateTimeRange
queryFactor
queryFactorIndex
mergeStatus
getConditionSet
getSingleCondition
refactorCsId
changeFactorWhenPartialAccept
checkPartialAcceptSc

Facet函数之间互相调用

在Facet合约中, address(this)返回的是钻石合约的地址, 因此可通过该地址调用其他Facet合约中的函数

部署

1. 部署标准DiamondFacetCut合约
2. 部署标准DiamondLoupe合约
3. 部署OwnershipFacet合约 (可选, 用于转移所有权)
4. 部署标准Diamond合约, 这里会携带第一步的DiamondFacetCut合约地址来创建我们的Diamond
5. 部署我们自己的业务Facet合约
6. 将各个业务Facet合约包含的函数通过diamondCut方法注册到Diamond中

升级

1. 脚本部署新的业务Facet合约
2. 脚本通过调用DiamondLoupe合约来获取Diamond合约目前的所有Facet的函数
3. 脚本比对新旧版本Diamond的facets, 新增的facet记为add, 新旧版本facet地址不同的记为replace, 旧版本中存在新版本不存在的记为remove, 最终拼接成一个交易数据,
4. 由治理合约来调用DiamondFacetCut, 可以在一个交易中完成Diamond的升级动作

使用的钻石代理标准合约库:

<https://github.com/mudgen/diamond-3>

ABI合并插件

<https://github.com/projectsophon/hardhat-diamond-abi>

hardhat钻石代理插件

<https://github.com/mudgen/diamond-3-hardhat>